

Project 1

Spartan-6 FPGA — DSP48A1 Slice

RTL Design and Functional Verification

Prepared by

Esraa Tarek Fouad Ahmed
Ahmed Mohamed Ahmed Mohamed

Tool: QuestaSim (functional simulation)

Table of Contents

1. Introduction	1
1.1 Project Flow	1
2. RTL Code — DSP48A1.v	2
3. Testbench Code — tb_DSP48A1.v	5
4. Simulation — QuestaSim	8
4.1 run.do Automation Script	8
4.2 Waveform & Transcript	8
5. Conclusion	9

1. Introduction

The DSP48A1 is the dedicated arithmetic slice found in Xilinx Spartan-6 FPGAs. Each slice combines an optional pre-adder/subtractor, an 18×18-bit two's-complement multiplier, a 48-bit post-adder/subtractor (ALU), and a rich set of pipeline registers and cascade paths (BCOUT/BCIN, PCOUT/PCIN). This structure lets a single hardened slice implement multiply-accumulate operations, FIR filter taps, and general arithmetic datapaths at high speed and low resource cost, instead of building the same logic from generic fabric.

The goal of this project is to build a synthesizable, parameterized behavioral model of the DSP48A1 slice in Verilog and verify its functional correctness against the documented operating modes using a self-checking testbench in QuestaSim.

1.1 Project Flow

- RTL modeling of the DSP48A1 slice in Verilog (parameterized register/pipeline stages).
- Self-checking testbench covering reset behavior and four representative datapaths (Path 1–4).
- Functional simulation in QuestaSim using a `run.do` automation script.

2. RTL Code — DSP48A1.v

The module below implements the DSP48A1 datapath: independently-enabled input registers (A0/A1, B0/B1, C, D, OPMODE), a pre-adder feeding the B1 register, the 18×18 multiplier with its M register, the X/Y/Z operand multiplexers driven by OPMODE, the carry-in register, and the final post-adder/subtractor feeding the P and carry-out registers. All sequential elements share the RSTTYPE parameter (synchronous or asynchronous reset) and individual clock-enables, matching the behavior described for the Xilinx primitive.

```
`timescale 1ns / 1ps module DSP48A1 #(      // -----
Parameters -----      parameter A0REG = 0,      parameter A1REG
= 1,      parameter B0REG = 0,      parameter B1REG = 1,      parameter
CREG = 1,      parameter DREG = 1,      parameter MREG = 1,      parameter
PREG = 1,      parameter CARRYINREG = 1,      parameter CARRYOUTREG = 1,
parameter OPMODEREG = 1,      parameter CARRYINSEL = "OPMODE5",
parameter B_INPUT = "DIRECT",      parameter RSTTYPE = "SYNC" ) (      //
----- Data Inputs -----      input wire [17:0] A,
input wire [17:0] B,      input wire [17:0] D,      input wire [47:0] C,
// ----- Cascade Inputs -----      input wire [17:0]
BCIN,      input wire [47:0] PCIN,      input wire CARRYIN,      // -----
----- Control Inputs -----      input wire [7:0] OPMODE,
input wire CLK,      // ----- Clock Enable Inputs -----
input wire CEA, CEB, CEC, CECARRYIN, CED, CEM, CEOPMODE, CEP,      // --
----- Reset Inputs -----      input wire RSTA, RSTB,
RSTC, RSTCARRYIN, RSTD, RSTM, RSTOPMODE, RSTP,      // -----
Data Outputs -----      output wire [17:0] BCOUT,      output
wire [47:0] PCOUT, P,      output wire [35:0] M,      output wire
CARRYOUT, CARRYOUTF ); // Internal Signals & Wires wire [17:0]
b_mux_out = (B_INPUT == "CASCADE") ? BCIN : B; reg [17:0] d_reg_out,
b0_reg_out, a0_reg_out; reg [47:0] c_reg_out; reg [7:0] opmode_reg_out;
reg [17:0] b1_reg_out, a1_reg_out; reg [35:0] m_reg_out; reg [47:0]
p_reg_out; reg cyi_reg_out, cyo_reg_out; wire [17:0] pre_adder_out;
wire [17:0] b1_mux_in; wire [35:0] mult_out; wire [47:0] c_mux_out; reg
[47:0] x_mux_out, z_mux_out; wire post_adder_cin, post_adder_cout; wire
[47:0] post_adder_out;
// Stage 1: Initial Pipeline Registers (A0, B0, D, C, OPMODE)

always @(posedge CLK or posedge RSTA) begin      if (RSTA
&& RSTTYPE == "ASYN") a0_reg_out <= 18'b0;      else if
(RSTA && RSTTYPE == "SYNC") a0_reg_out <= 18'b0;      else
if (CEA) a0_reg_out <= A; end wire [17:0] a0_mux_out =
A0REG ? a0_reg_out : A;

always @(posedge CLK or posedge RSTB) begin      if (RSTB
&& RSTTYPE == "ASYN") b0_reg_out <= 18'b0;      else if
(RSTB && RSTTYPE == "SYNC") b0_reg_out <= 18'b0;      else
if (CEB) b0_reg_out <= b_mux_out; end wire [17:0]
b0_mux_out = B0REG ? b0_reg_out : b_mux_out;

always @(posedge CLK or posedge RSTD) begin      if (RSTD
&& RSTTYPE == "ASYN") d_reg_out <= 18'b0;      else if
(RSTD && RSTTYPE == "SYNC") d_reg_out <= 18'b0;      else
if (CED) d_reg_out <= D; end wire [17:0] d_mux_out = DREG
? d_reg_out : D;

always @(posedge CLK or posedge RSTC) begin      if (RSTC
&& RSTTYPE == "ASYN") c_reg_out <= 48'b0;      else if
(RSTC && RSTTYPE == "SYNC") c_reg_out <= 48'b0;      else
if (CEC) c_reg_out <= C; end assign c_mux_out = CREG ?
c_reg_out : C;

always @(posedge CLK or posedge RSTOPMODE) begin      if (RSTOPMODE
&& RSTTYPE == "ASYN") opmode_reg_out <= 8'b0;      else if
```

```

(RSTOPMODE && RSTTYPE == "SYNC") opmode_reg_out <= 8'b0;      else
if (CEOPMODE) opmode_reg_out <= OPMODE; end wire [7:0] op_mux_out
= OPMODEREG ? opmode_reg_out : OPMODE;

// Pre-Adder / Subtractor & Stage 2 Registers (A1, B1) assign pre_adder_out =
op_mux_out[6] ? (d_mux_out - b0_mux_out) : (d_mux_out + b0_mux_out); assign b1_mux_in =
op_mux_out[4] ? pre_adder_out : b0_mux_out;

always @(posedge CLK or posedge RSTA) begin      if (RSTA
&& RSTTYPE == "ASYNC") a1_reg_out <= 18'b0;      else if
(RSTA && RSTTYPE == "SYNC") a1_reg_out <= 18'b0;      else
if (CEA) a1_reg_out <= a0_mux_out; end wire [17:0]
a1_mux_out = A1REG ? a1_reg_out : a0_mux_out;

always @(posedge CLK or posedge RSTB) begin      if (RSTB
&& RSTTYPE == "ASYNC") b1_reg_out <= 18'b0;      else if
(RSTB && RSTTYPE == "SYNC") b1_reg_out <= 18'b0;      else
if (CEB) b1_reg_out <= b1_mux_in; end wire [17:0]
b1_mux_out = B1REG ? b1_reg_out : b1_mux_in;

// Multiplier & M Register assign
mult_out = a1_mux_out * b1_mux_out;

always @(posedge CLK or posedge RSTM) begin      if (RSTM
&& RSTTYPE == "ASYNC") m_reg_out <= 36'b0;      else if
(RSTM && RSTTYPE == "SYNC") m_reg_out <= 36'b0;      else
if (CEM) m_reg_out <= mult_out; end
wire [35:0] m_mux_out = MREG ? m_reg_out :
mult_out; assign M = m_mux_out; // 48-bit X and Z

Multiplexers

wire [47:0] concat_dab = {d_mux_out[11:0], a1_mux_out[17:0], b1_mux_out[17:0]};
always @(*) begin      case
(op_mux_out[1:0])          2'b00:
x_mux_out = 48'b0;
2'b01: x_mux_out = {12'b0, m_mux_out};
2'b10: x_mux_out = P;
2'b11: x_mux_out = concat_dab;
default: x_mux_out = 48'b0;
endcase end

always @(*) begin      case
(op_mux_out[3:2])          2'b00:
z_mux_out = 48'b0;
2'b01: z_mux_out = PCIN;
2'b10: z_mux_out = P;
2'b11: z_mux_out = c_mux_out;
default: z_mux_out = 48'b0;
endcase end

// Carry-In Logic (CYI) wire cin_mux_out = (CARRYINSEL == "OPMODE5")
? op_mux_out[5] : CARRYIN;

always @(posedge CLK or posedge RSTCARRYIN) begin      if
(RSTCARRYIN && RSTTYPE == "ASYNC") cyi_reg_out <= 1'b0;      else
if (RSTCARRYIN && RSTTYPE == "SYNC") cyi_reg_out <= 1'b0;
else if (CECARRYIN) cyi_reg_out <= cin_mux_out; end assign
post_adder_cin = CARRYINREG ? cyi_reg_out : cin_mux_out;

// Post-Adder / Subtractor & P / Carry-Out Registers assign {post_adder_cout,
post_adder_out} = op_mux_out[7] ?
(z_mux_out - (x_mux_out + post_adder_cin)) :
(z_mux_out + x_mux_out + post_adder_cin);

```

```
always @(posedge CLK or posedge RSTP) begin    if (RSTP
&& RSTTYPE == "ASYNC") p_reg_out <= 48'b0;    else if
(RSTP && RSTTYPE == "SYNC") p_reg_out <= 48'b0;    else
if (CEP) p_reg_out <= post_adder_out; end wire [47:0]
p_final = PREG ? p_reg_out : post_adder_out;

always @(posedge CLK or posedge RSTCARRYIN) begin    if
(RSTCARRYIN && RSTTYPE == "ASYNC") cyo_reg_out <= 1'b0;    else
if (RSTCARRYIN && RSTTYPE == "SYNC") cyo_reg_out <= 1'b0;
else if (CECARRYIN) cyo_reg_out <= post_adder_cout; end wire
carry_final = CARRYOUTREG ? cyo_reg_out : post_adder_cout; //
Output Assignments

assign P = p_final; assign
PCOUT = p_final; assign BCOUT
= b1_mux_out; assign CARRYOUT
= carry_final; assign
CARRYOUTF = carry_final;

endmodule
```

3. Testbench Code — tb_DSP48A1.v

The testbench instantiates the DSP48A1 with the default parameter set (A1REG/B1REG/C/D/M/P/ carry registers enabled, synchronous reset) and drives a 100 MHz clock. It self-checks five scenarios: (1) global reset — all outputs must go to zero; (2)–(4) three representative OPMODE paths exercising the pre-adder, multiplier, and post-adder feedback/cascade behavior; and (5) a path that exercises the carry chain together with the PCIN cascade input. Each scenario prints a PASS/FAIL verdict to the transcript.

```
`timescale 1ns / 1ps module
tb_DSP48A1; // Signals &
Testbench Variables

reg [17:0] A, B, D, BCIN; reg [47:0] C, PCIN; reg CARRYIN,
CLK; reg [7:0] OPMODE; reg CEA, CEB, CEC, CECARRYIN, CED,
CEM, CEOPMODE, CEP; reg RSTA, RSTB, RSTC, RSTCARRYIN, RSTD,
RSTM, RSTOPMODE, RSTP;

wire [17:0] BCOUT; wire
[47:0] PCOUT, P; wire
[35:0] M; wire CARRYOUT,
CARRYOUTF;

// Variables to store past values for Path 3
checking reg [47:0] past_P; reg past_CARRYOUT;

// Device Under Test (DUT) Instantiation
// Instantiated with default parameters as specified
DSP48A1 #(
    .A0REG(0), .A1REG(1), .B0REG(0), .B1REG(1),
    .CREG(1), .DREG(1), .MREG(1), .PREG(1),
    .CARRYINREG(1), .CARRYOUTREG(1), .OPMODEREG(1),
    .CARRYINSEL("OPMODE5"), .B_INPUT("DIRECT"), .RSTTYPE("SYNC")
) uut (
    .A(A), .B(B), .D(D), .C(C),
    .BCIN(BCIN), .PCIN(PCIN), .CARRYIN(CARRYIN),
    .OPMODE(OPMODE), .CLK(CLK),
    .CEA(CEA), .CEB(CEB), .CEC(CEC), .CECARRYIN(CECARRYIN),
    .CED(CED), .CEM(CEM), .CEOPMODE(CEOPMODE), .CEP(CEP),
    .RSTA(RSTA), .RSTB(RSTB), .RSTC(RSTC), .RSTCARRYIN(RSTCARRYIN),
    .RSTD(RSTD), .RSTM(RSTM), .RSTOPMODE(RSTOPMODE), .RSTP(RSTP),
    .BCOUT(BCOUT), .PCOUT(PCOUT), .P(P), .M(M),
    .CARRYOUT(CARRYOUT), .CARRYOUTF(CARRYOUTF));

// Clock Generation
initial CLK = 0;
always #5 CLK = ~CLK; // 100MHz clock

// Stimulus Generation (Initial
Block) initial begin //
Initialize Inputs
    A = 0; B = 0; C = 0; D = 0;
    BCIN = 0; PCIN = 0; CARRYIN = 0; OPMODE = 0;
    CEA = 0; CEB = 0; CEC = 0; CECARRYIN = 0; CED = 0; CEM = 0; CEOPMODE = 0; CEP = 0;
    RSTA = 0; RSTB = 0; RSTC = 0; RSTCARRYIN = 0; RSTD = 0; RSTM = 0; RSTOPMODE = 0; RSTP =
0;

// Step 2.1: Verify Reset Operation
$display("Starting Step 2.1: Verify Reset Operation");

// Assert all active-high resets
```

```

RSTA = 1; RSTB = 1; RSTC = 1; RSTCARRYIN = 1;
RSTD = 1; RSTM = 1; RSTOPMODE = 1; RSTP = 1;

// Drive remaining inputs with arbitrary values
A = $random; B = $random; C = $random; D = $random;

// Wait for the negative edge of the clock
@(negedge CLK);

// Self-checking condition to verify outputs are zero      if (BCOUT === 0 && PCOUT ===
0 && P === 0 && M === 0 && CARRYOUT === 0 && CARRYOUTF === 0)      $display("-> PASS:
Reset successful. All outputs are zero.");      else      $display("-> FAIL: Outputs are
not zero after reset.");

// Deassert resets and assert clock enables
RSTA = 0; RSTB = 0; RSTC = 0; RSTCARRYIN = 0;
RSTD = 0; RSTM = 0; RSTOPMODE = 0; RSTP = 0;
CEA = 1; CEB = 1; CEC = 1; CECARRYIN = 1;
CED = 1; CEM = 1; CEOPMODE = 1; CEP = 1;

// Step 2.2: Verify DSP Path 1

$display("Starting Step 2.2: Verify DSP Path 1");
OPMODE = 8'b11011101;
A = 20; B = 10; C = 350; D = 25;
BCIN = $random; PCIN = $random; CARRYIN = $random;

// Wait for four negative clock edges
repeat(4) @(negedge CLK);

if (BCOUT === 18'hf && M === 36'h12c && P === 48'h32 && PCOUT ===
48'h32      && CARRYOUT === 0 && CARRYOUTF === 0)      $display("->
PASS: Path 1 outputs match expected values.");      else
$display("-> FAIL: Path 1 outputs incorrect. P=%h", P);

// Step 2.3: Verify DSP Path 2

$display("Starting Step 2.3: Verify DSP Path 2");
OPMODE = 8'b00010000;
A = 20; B = 10; C = 350; D = 25;
BCIN = $random; PCIN = $random; CARRYIN = $random;

// Wait for three negative clock edges
repeat(3) @(negedge CLK);

if (BCOUT === 18'h23 && M === 36'h2bc && P === 0 && PCOUT === 0
&& CARRYOUT === 0 && CARRYOUTF === 0)      $display("-> PASS: Path
2 outputs match expected values.");      else      $display("->
FAIL: Path 2 outputs incorrect. P=%h, M=%h", P, M);

// Store past values for Path 3
verification      past_P = P;      past_CARRYOUT
= CARRYOUT;

// Step 2.4: Verify DSP Path 3
$display("Starting Step 2.4: Verify DSP Path 3");
OPMODE = 8'b00001010;
A = 20; B = 10; C = 350; D = 25;
BCIN = $random; PCIN = $random; CARRYIN = $random;

// Wait for three negative clock edges
repeat(3) @(negedge CLK);

```

```
    if (BCOUT === 18'ha && M === 36'hc8 && P === past_P && PCOUT === past_P
        && CARRYOUT === past_CARRYOUT && CARRYOUTF === past_CARRYOUT)
$display("-> PASS: Path 3 outputs match expected values (Feedback retained).");
else
    $display("-> FAIL: Path 3 outputs incorrect.");

// Step 2.5: Verify DSP Path 4
$display("Starting Step 2.5: Verify DSP Path 4");
OPMODE = 8'b10100111;
A = 5; B = 6; C = 350; D = 25; PCIN = 3000;
BCIN = $random; CARRYIN = $random;

// Wait for three negative clock edges
repeat(3) @(negedge CLK);

    if (BCOUT === 18'h6 && M === 36'h1e && P === 48'hfe6fffec0bb1 && PCOUT ===
48'hfe6fffec0bb1 && CARRYOUT === 1 && CARRYOUTF === 1)
        $display("-> PASS: Path 4 outputs match expected values.");
    else
        $display("-> FAIL: Path 4 outputs incorrect. P=%h", P);

// End Simulation
$display("Simulation Complete.");
$finish; end

endmodule
```


4. Simulation — QuestaSim

4.1 run.do Automation Script

Simulation is fully scripted: the script builds the work library, compiles the RTL and testbench, launches the simulator, adds every signal in the testbench scope to the waveform, runs to completion, and zooms the wave window to fit.

```
vlib work vmap work work vlog -work work
DSP48A1.v vlog -work work tb_DSP48A1.v vsim -
voptargs=+acc work.tb_DSP48A1 add wave -
position insertpoint sim:/tb_DSP48A1/* run -
all wave zoom full
```

4.2 Waveform & Transcript

The capture below shows the full QuestaSim session after `run -all`: the Objects/Wave panes with every DUT signal, and the Transcript pane confirming the simulation ran to completion (`$finish` at 147 ns) with no unexpected errors reported.

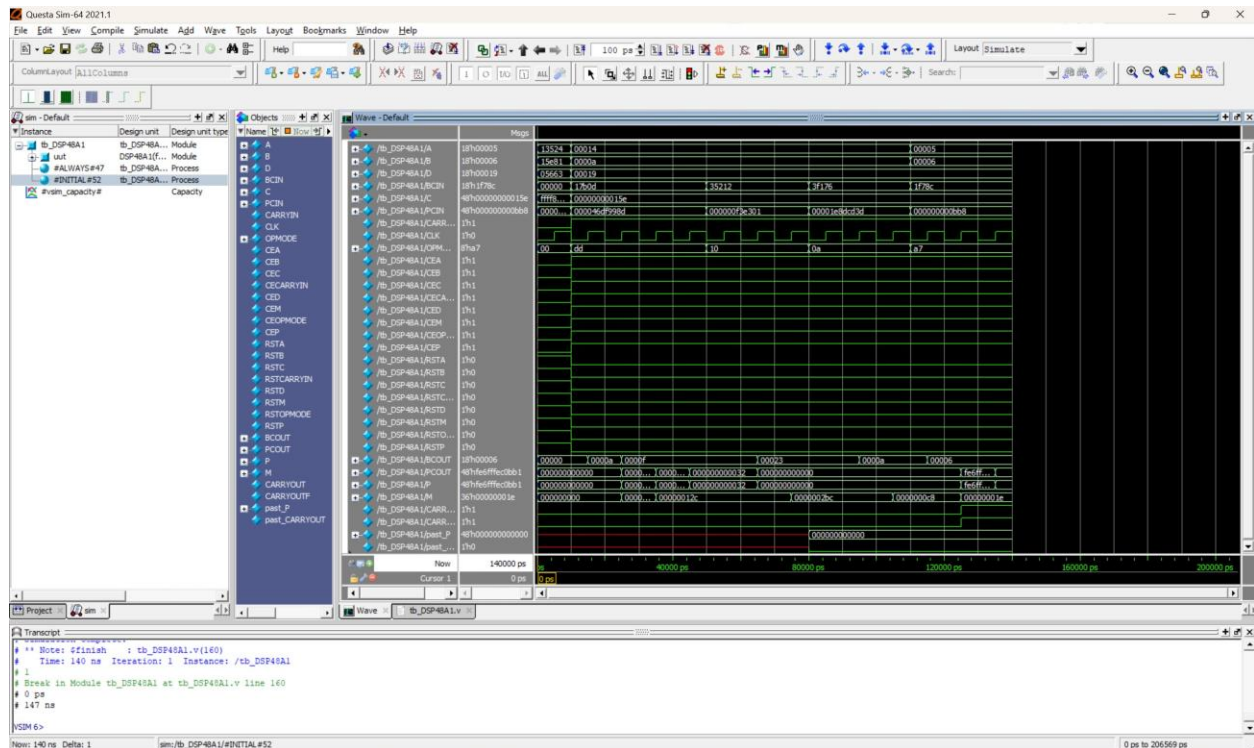


Figure 4.1 — QuestaSim waveform and transcript for `tb_DSP48A1`.

5. Conclusion

The DSP48A1 behavioral model was implemented and functionally verified in QuestaSim. The self-checking testbench confirmed correct reset behavior and correct results across four representative OPMODE paths (pre-adder, multiplier, post-adder feedback, and carry/cascade), with the simulation running to completion and no failing checks reported in the transcript. The project demonstrates RTL capture and simulation-based verification of a complex, parameterized arithmetic datapath.